

FRAUD DETECTION PLATFORM

Teknik Dokümantasyon

IEEE-CIS Fraud Detection | Adım 8–10

1. Genel Bakış

Bu platform, Adım 1–7'de geliştirilen IEEE-CIS Fraud Detection analiz altyapısını production-ready bir FastAPI servisi olarak sunar. Temel hedef: gelen bir transaction'ın fraud olasılığını çok katmanlı anomali skorlaması, iş kuralları ve LLM destekli açıklanabilirlik ile değerlendirmektir.

Temel Özellikler

- Hibrit scoring: Adım 7 parquet cache + canlı Adım 3–7 pipeline
- Sinyal bazlı multi-agent orchestration (7 farklı strateji)
- RAG tabanlı açıklanabilirlik: FAISS vector search + local LLM
- YAML tabanlı configurable kural motoru + tam audit trail
- Toplu (batch) scoring, /score/batch endpoint
- Dependency injection (dependency-injector, 5 tasarım deseni)

Teknoloji Yığını

Katman	Teknoloji
API	FastAPI + Uvicorn
Agent Orchestration	Özel BaseAgent ABC + AgentMessage
RAG — Embedding	sentence-transformers/all-MiniLM-L6-v2 (CPU)
RAG — Vector Store	FAISS (IndexFlatIP, cosine similarity)
RAG — LLM	LM Studio — google/gemma-4-e4b (local)
Anomali Tespiti	Isolation Forest (scikit-learn, Adım 4)
Kural Motoru	YAML tabanlı configurable engine (Adım 7)
Dependency Injection	dependency-injector
Veri	pandas + pyarrow (parquet)

2. Sistem Mimarisi

Platform üç ana katmandan oluşur: API katmanı (FastAPI), agent orchestration katmanı ve servis katmanı (RAG, pipeline, cache).

API → Agent → Servis Akışı

```
HTTP İsteği → FastAPI Router
    → Depends(get_orchestrator) → Container
    → OrchestratorAgent.execute()
        → _detect_strategy() # sinyal analizi
        → ScoringAgent # her zaman
        → RuleAgent # score >= 0.3 veya strateji !=
rule_based
        → ExplanationAgent # score >= 0.5 veya güçlü sinyal
            → RAG Pipeline
                → embed_query() → FAISS search()
                → build_rag_prompt() → LM Studio
```

Hibrit Scoring Pipeline

Her /score isteğinde PipelineService şu sırayı izler:

- TransactionID Adım 7 parquet cache'inde aranır (590k satır, startup'ta yüklenir)
- Cache hit → skorlar anında döndürülür (pipeline_source: "cache")
- Cache miss → Adım 3–7 canlı çalıştırılır (pipeline_source: "pipeline")
- Pipeline başarısız → ham transaction değerleri kullanılır (pipeline_source: "fallback")

3. Agent Sistemi (Adım 9)

OrchestratorAgent — Strateji Sistemi

Adım 5'teki AUC ağırlıkları kullanılarak hangi sinyal katmanının baskın olduğu tespit edilir. Basit eşik kararı yerine, her strateji farklı agent kombinasyonu çalıştırır.

Strateji	Tetikleyici	Açıklama
statistical	column_score_mean > 0.60	İstatistiksel özellik sapması (AUC 0.73, %37 ağırlık)
statistical_new_card	column yüksek + entity < 0.30	Düşük geçmişli kart — yeni/çalıntı kart pattern
behavioral	entity_score > 0.50	Entity davranışsal sapması (AUC 0.53)
behavioral_multivariate	entity + mv yüksek	Hesap ele geçirme pattern — çift anomali
multivariate	multivariate_score > 0.55	Isolation Forest joint distribution anomalisi
temporal	temporal_score > 0.50	Zaman pattern sapması (AUC 0.52)

rule_based	Model sinyalleri zayıf	Yalnızca iş kuralları tetiklenir
------------	------------------------	----------------------------------

AgentMessage — Agent-to-Agent İletişim

Tüm agent'lar arasındaki iletişim AgentMessage nesnesi üzerinden yürütülür. Her mesaj task, payload, sender, receiver, result ve error alanlarını taşır. Orchestrator delegate() metodu ile hata yalıtımı sağlar — bir agent başarısız olursa diğerleri çalışmaya devam eder.

4. RAG Pipeline (Adım 8)

Knowledge Base

24 döküman, 3 grupta organize edilmiştir:

Grup	ID Aralığı	İçerik
Sentetik fraud kuralları	KB001–KB015	Velocity, gece, email domain, kart tipi, güvenilir entity politikaları
Model sinyal dökümanları	KB016–KB021	Adım 5 AUC bulgularına dayalı: column_score_mean (0.73), column_score_max (0.70), Isolation Forest (0.64), entity (0.53), temporal (0.52)
Feature kombinasyonları	KB022–KB024	Adım 2 analizinden: column+gece+velocity, dual amount anomaly, yeni kart pattern

Retrieval Akışı

- transaction_to_narrative() — ham veriyi deterministik doğal dile çevirir (LLM değil, kod)
- Strateji context'i narrative'e eklenir (Detection strategy: ...)
- embed_query() — sentence-transformers ile 384 boyutlu vektör
- FAISS search() — cosine similarity, top-3 KB dökümanı
- build_rag_prompt() — narrative + KB içerikleri + driver features → LLM prompt
- query_llm() — LM Studio HTTP call, timeout + fallback

RAG Kalite Kanıtı

Strateji sistemi ile RAG retrieval kalitesi iyileştirilmiştir. statistical_new_card stratejisi seçildiğinde KB024 (Weak Entity Signal with Strong Column Signal) otomatik olarak en yüksek skorda gelir — bu, modelin kendi bulgularıyla tutarlı dökümanları bulduğunun somut kanıtıdır.

5. API Endpoint'leri (Adım 10)

GET /health

Platform sađlık durumu. LLM bađlantısı, FAISS index durumu ve KB döküman sayısını döndürür.

POST /score

Tek transaction fraud skoru. Hibrit pipeline (cache → Adım 3–7 → fallback). Response'da fraud_score, risk_level, strategy, dominant_signals, pipeline_source bulunur.

risk_level deđerleri: low (<0.3), medium (0.3–0.5), high (0.5–0.7), very_high (>0.7)

POST /score/batch

Max 1000 transaction'lı toplu skorlar. Sonuçta risk_summary dağılımı (low/medium/high/very_high sayısı) döner. Batch'te default mode score_only'dir, RAG çağrısı yapılmaz.

POST /explain

RAG + LLM açıklaması. Narrative, driver_features, retrieved_docs (FAISS sonuçları), LLM explanation ve rule_audit_trail döner. Orchestrator üzerinden geçtiđi için strateji ve dominant_signals da bulunur.

POST /rules/evaluate

YAML kural motoru deđerlendirmesi. 10 kuralın tamamı için tam audit trail döner: triggered/skipped, multiplier, severity, flag ve açıklama.

POST /rag/query

Serbest metin ile knowledge base sorgusu. Doğrudan FAISS search + LLM yanıtı — transaction olmadan da sorgulanabilir.

6. Kural Motoru (Adım 7 Entegrasyonu)

10 iş kuralı rules/fraud_rules.yaml dosyasında tanımlıdır. YAML format: id, conditions (field/op/value), action (multiplier/severity/flag). Conflict resolution: max_multiplier stratejisi.

Kural	Severity	Multiplier	Açıklama
R001	HIGH	1.50x	Gece + yüksek tutar + riskli email
R002	HIGH	1.45x	Velocity spike + yüksek tutar
R005	HIGH	1.40x	Amount Z-score >= 3.0
R008	HIGH	1.40x	Gece velocity spike
R003	MEDIUM	1.35x	Riskli card4 tipi
R009	MEDIUM	1.35x	Riskli email + velocity

R004	MEDIUM	1.30x	Riskli card6 + ürün
R010	MEDIUM	1.30x	Tutar >= \$2000
R006	MEDIUM	1.25x	Hafta sonu + yüksek tutar + riskli ürün
R007	LOW	0.75x	Güvenilir entity (azaltıcı)

7. Dependency Injection (Bonus)

containers.py — dependency_injector kütüphanesi ile 5 tasarım deseni uygulanmıştır:

Desen	Uygulama
Singleton	FAISS index, embedding model, tüm agent'lar — startup'ta bir kez oluşturulur, tüm istekler aynı instance'ı paylaşır
Factory	AgentMessage — her request'te yeni instance
Configuration	Tüm ayarlar config.settings'ten beslenir, test ortamında override edilebilir
Strategy	llm_model config değeri değiştirilerek farklı LLM'e geçiş yapılabilir
Repository	kb_docs — knowledge base için repository rolü, load_knowledge_base() ile soyutlanmış

8. Kurulum ve Çalıştırma

Gereksinimler

- Python 3.9+
- LM Studio (google/gemma-4-e4b, port 1234)
- Adım 7 parquet çıktısı (outputs/step7/df_rules_train.parquet)
- Adım 4 joblib modelleri (pipeline/models/)

Kurulum Adımları

```
pip install -r requirements.txt
# config.py'de STEP7_OUTPUT_PATH'i ayarla
# LM Studio'da Gemma modelini yükle ve server'ı başlat
python -m uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

Test

```
python test_api.py
```

Test senaryoları: /health, HIGH/LOW RISK skorlama, CACHE HIT (bilinen ID), PIPELINE FALLBACK (ID=9999999), rules audit trail, batch scoring, explain (LLM), RAG query.

9. Doğrulama ve Test Çıktıları

Çalışan Sistem Kanıtları

Test	Sonuç	Önemli Detay
/health	200 OK	kb_doc_count: 24, llm_available: true, faiss_loaded: true
/score HIGH RISK	fraud_score: 0.78, very_high	strategy: statistical_new_card, 3 agent çalıştı
/score LOW RISK	fraud_score: 0.12, low	Yalnızca ScoringAgent — gereksiz RAG yok
/score CACHE HIT	pipeline_source: cache	Adım 7 parquet'ten anında döndü
/score FALLBACK	pipeline_source: fallback	Bilinmeyen ID — ham değerler kullanıldı
/rules/evaluate	R001,R005,R008,R009 tetiklendi	max_multiplier: 1.5, 10 kuralın tam audit trail'i
/score/batch (3 tx)	very_high / low / high	risk_summary dağılımı doğru
/explain	strategy: statistical_new_card	KB024 en yüksek skorla geldi — RAG tutarlı
/rag/query	KB002, KB001, KB022	KB022 yeni feature kombinasyon dökümanı — RAG iyileşmesi